

p1_03_stack_heap.cpp ·内存结构图

数据源：code/outputs/p1_03_stack_heap.txt（Apple clang 21 / arm64 / macOS 的存档实跑输出）。地址每次运行都不同 —— 看相对关系，不看具体数值。

栈（自动回收） 堆（要手动 delete） 全局 / 静态区 代码段 本次没用到的位置

① 栈 [stack]

高地址在上：main 的栈帧先占好位置，它调用的 level(0..3) 依次落在更低的地址上。

main 栈帧0x16f4ca430 - 0x16f4ca480（80 字节）head / a / b / c / arr[4] / scoped —— 第 2 页逐格画

level(0) 的 local&local = 0x16f4ca3e8

level(1) 的 local&local = 0x16f4ca3b8

level(2) 的 local&local = 0x16f4ca388

level(3) 的 local&local = 0x16f4ca358

地址由高到低

每往下一层低 0x30 字节 = 一个 level 栈帧；level() 返回时这 4 帧一次性回收，没有任何 delete

栈 0x16f4ca3e8 ↔ 堆 / 静态区 0x100eadba0：相差 约 1.7 GiB，是两段互不相干的区域，不是挨着的两块；同一份代码换到 Linux x86-64 / GCC 16.2 上是 约 33 TiB —— 差几个数量级这件事是稳的，具体数值不是。

② 堆（new / malloc 的区）

按地址从小到大摆好这三块：第 3 次 new 反而落到了最低地址 —— 分配顺序 ≠ 地址顺序，间隔和顺序都由分配器定，不是语言保证。

head 变量（本身在栈上）0x16f4ca450

next

Node value=20x100eada10第 3 次 new

Node value=00x100eadba0第 1 次 new

Node value=10x100eadbb0第 2 次 new

存档这次运行：node0 0x100eadba0、node1 0x100eadbb0（间隔 0x10），node2 0x100eada10 比 node1 低 0x1a0；本机 glibc 则是间隔 0x20 且一路递增。两边的共同点只有一个：每块 Node 都是 16 字节，谁 new 谁 delete。

③ 全局 / 静态区

生命周期 = 整个进程，不受函数返回影响

static int g_counter = 0x10093c000

和堆同在 0x10093... 那一段，离栈 约 1.7 GiB

④ 代码段（可执行文件本体，只读）

main / level / Node 的机器码躺在地址最低的一端：new 和 delete 都是这里发出的指令，指令本身不占栈也不占堆。

这道例题的四段路 按源码顺序读，每一段都对应上面的一块区域

源码（code/p1_03_stack_heap.cpp）	这一段想让你看见什么
level(0) → level(3)（19 – 20 行）	栈向下长：每层一个固定栈帧，返回时整段回收
g_counter / a b c / arr（21 – 31 行）	三段区间互不相干；数组元素必须连续，独立变量不保证
new Node x3(34 – 39 行)	堆的地址和顺序由分配器定，new 只保证「给你 16 字节」
scoped / reused / delete（44 – 58 行）	栈自动复用、堆手动释放：谁 new 谁 delete

第 1 页小结

1. 栈向下长，每层一个固定大小的栈帧：存档里四层 level 每层稳差 0x30 字节；函数返回时整段一次性回收，不需要写任何 delete。

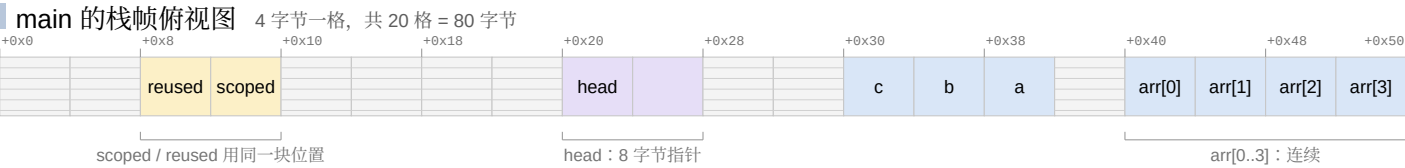
2. 堆的地址、间隔、甚至先后顺序都不是语言保证：存档这次运行里第 3 次 new 落到了最低地址，所以只记住「谁 new 谁 delete」，别记住地址长什么样。

3. 栈、堆、静态区是三段互不相干的区域，它们之间唯一的接口是「指针 + 生命周期」——内存出问题，基本都出在这条接口上（第 3 页）。

p1_03_stack_heap.cpp ·内存结构图（2/3）· main的栈帧

同一个函数里的变量全在 main 的栈帧里。下面从 0x16f4ca430 起、4 字节一格画出来，格子里的地址来自存档的同一运行。

普通局部变量 作用域一结束就被复用 指针变量（指向堆） 这次没打印到的位置



- arr[0..3] 四格紧挨着、步长恰好 4 = sizeof(int)：数组是一个完整的对象，元素必须连续存放。
- a / b / c 也挨着，但谁在高位完全由编译器决定：这份存档是递减的（0x16f4ca468 > 0x16f4ca464 > 0x16f4ca460），本机 GCC 却是递增的（a < b < c）—— 别按声明顺序推地址。
- head 占两格：它是个 8 字节指针，自己老老实实待在栈上，指向的对象在堆上 —— 「变量」和「变量指向的东西」不一定在同一个区。
- 灰色格子是这次运行没打印到的位置：对齐和调用约定要给临时值留空档，所以栈帧里总有不属于任何变量的字节。
- reused 落在 scoped 用过的位置上（存档里两者相邻 4 字节，本机 GCC 实测地址完全相同）：作用域一结束，那块栈空间马上被别人接手。

递归：每层一个固定大小的栈帧 同一个 level() 反复调用，地址一路往下走

调用层	&local	与上一层差	说明
level(0)	0x16f4ca3e8	.	最先调用，地址最高
level(1)	0x16f4ca3b8	-0x30	低一个栈帧
level(2)	0x16f4ca388	-0x30	低一个栈帧
level(3)	0x16f4ca358	-0x30	低一个栈帧

- 每调用一层，栈指针往下走 0x30 字节：一个 int + 保存的返回地址 / 寄存器 + 对齐。
- 动手改一改：给 level() 加一个 char buff[64]，本机实测每层差从 0x30 变成 0x70 = 112 字节 —— 栈帧大小 = 局部变量之和（对齐之后）。
- 四层一共才 0xc0 字节；把深度换成十多万量级就是栈溢出（第三阶段例题 3-3 的伏笔）。

生命周期：作用域一结束，地址就被接管



- 栈的「释放」不是清空内存，只是把栈指针挪回去：值还躺在那儿，指针还在你手上，那就是野指针。
- 同一份代码换到本机 GCC，scoped 和 reused 实测落在完全相同的地址 —— 复用不是巧合，是规则。

动手改一改 先预测，再跑，最后回到这张图上对一对

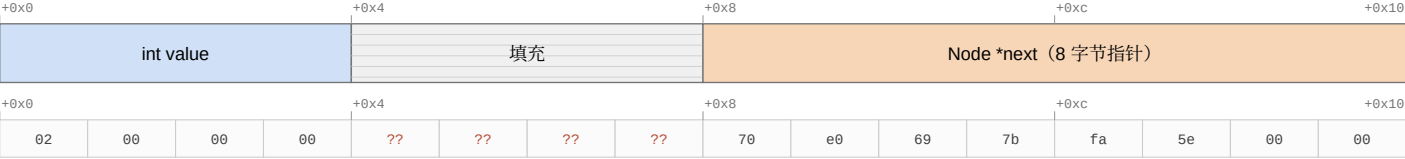
- 给 level() 加一个 char buff[64]：先算出每层差会变成多少（提示：向上取整到 16 的倍数），再跑 —— 本机实测是 0x70 = 112 字节。
- 把三次 new Node 改成 new Node[3]：堆上从 3 次分配变成 1 次、地址一定连续，释放也从 3 次 delete 变成 1 次 delete[] —— 这是唯一能「保证连续」的写法。
- 把 a / b / c 的声明顺序改一改，看地址跟不跟着换：验证「顺序由编译器定」，而不是「按声明顺序排」。

p1_03_stack_heap.cpp ·内存结构图（3/3）·堆上的 Node 与释放

堆这一侧的三个问题：一个 Node 占多少字节、三者怎么连起来、谁负责把它还回去。

数据成员 对齐填充（内容未定义） 指针字段 堆上的对象

Node 的 16 字节 8 字节的数据 + 8 字节的代价（存档与本机实测一致）



下排是本机 dump 出来的 16 个原始字节（小端）：02 00 00 00 是 value，紧跟着的 8 个字节是指向下一节点的地址；中间 4 个 ?? 是填充，内容未定义，这次刚好是 00。

- int value 在偏移 0；next 是 8 字节指针、要求 8 对齐，所以 4.7 只能当填充 —— 和例题 1-2 的填充是同一件事，只不过这次它让每个 Node 从 12 字节变成 16 字节。
- 结构体布局由 ABI 定，跨平台反而很稳：本机 GCC 与存档的 clang 都是 16 字节、value 在 0、next 在 8。

分配 → 链表 → 释放：每一步都不按地址来 存档这一次运行的完整顺序

步骤	发生了什么	地址
new #1	node0 : value=0, next=nullptr (head 当时还是空)	0x100eadba0
new #2	node1 : value=1, next=node0	0x100eadbb0
new #3	node2 : value=2, next=node1, 地址反而最低	0x100eada10
delete #1	先存下 head->next, 再 delete	0x100eada10
delete #2	head 前移到 node1, 重复上一步	0x100eadbb0
delete #3	head 变成 nullptr, 循环结束	0x100eadba0

```
while (head) { Node *next = head->next; delete head; head = next; }
```

- 必须先存 next：delete head 之后再读 head->next 就是读已释放内存（ASan 会报 heap-use-after-free，第三阶段例题 3-4）。
- 释放顺序 2 → 1 → 0，正好是 new 的倒序；栈上那些变量不用管，main 一返回全消失，堆上这三个必须有人 delete —— 漏一个就是泄漏。

换一台机器复核：结论稳、数值不稳 左列是仓库存档（macOS / arm64），右列是本机跑 probe_p1_03.cpp 的结果

观察项	macOS / Apple clang 21（存档）	Linux x86-64 / GCC 16.2（本机）
递归每层差	0x30	0x30
加 char buf[64] 后每层差	未测	0x70
Node 大小 / 偏移	16 字节, value@0、next@8	16 字节, value@0、next@8
三块堆的间隔与顺序	0x10, 第 3 块回到更低地址	0x20, 单调递增
a / b / c 地址顺序	递减（a 最高）	递增（a 最低）
scoped 与 reused	相邻 4 字节	地址完全相同
栈 ↔ 静态区距离	约 1.7 GiB	约 33 TiB

- 两列里唯一稳的是「结构」：栈向下、每层一个固定栈帧、Node 16 字节、指针在栈而对象在堆。
- 数值每次运行都在变（地址、间隔、距离都会被 ASLR 和分配器改掉），所以只能记规律、不能记数字 —— 这是这道例题最实用的一条。

栈 vs 堆：一张对照表 本例题标题里的「本质区别」就是这六行

对比项	栈	堆
空间从哪来	进入函数时由编译器预留	new / malloc 向分配器申请
谁释放、什么时候	运行时自动：作用域结束 / 函数返回	你：delete / free, 可提前也可延后
释放晚了	——	泄漏：长跑程序内存只涨不落
释放早了 / 越界	踩到相邻栈变量，现象随运行改变	野指针、double free（例题 3-4、3-5）
地址规律	必然向下，每层一个固定大小的帧	由分配器决定：可高可低、间隔不定
容量上限	通常 8 MB 量级（本机 ulimit -s = 8192）	受物理内存 + 交换空间限制

三条结论

1. 栈和堆的差别不是「快慢」，是「谁负责」：栈由编译器和运行时按作用域自动回收，堆必须由你 delete —— 漏了是泄漏，早删了是野指针。

2. 地址的方向与顺序都不是语言保证：栈一定向下长、每层一个固定栈帧；堆的地址和间隔看分配器，甚至可能像存档里那样「后分配的反而更低」。

3. 知道这些是为了做对两件事：跨语言传结构体时按 ABI 而不是靠猜来算大小；写宿主层接口时先想清楚「谁拥有、活到什么时候」，再动手写代码。